

情報実験I

ハードウェアを意識した
“効率的なプログラム”の作成
—組込みシステム開発にむけて—

担当：高橋

組込みシステムとは

- ・ コンピュータを内臓するシステム（PCなどのコンピュータを除く）
- ・ 家電，AV機器，ゲーム機，PC周辺機器，通信端末，自動販売機，自動車，ロボット，エレベータ，医用機器，ロケット等など，あらゆるシステムが該当．
- ・ 組込みシステムはSEが開発

組込みシステム向けソフトウェア

- ・ 限られた資源を用いて要求性能をいかに満たすかが問われる.
- ・ ハードウェアを意識したプログラミングが必要(特長を生かすプログラミング)
- ・ メモリ使用量, データの入出力時間, 処理時間

目的

- ・ PC(ホームディレクトリまでのネットワークを含む)を組み込みシステムとみなし, 特定の処理を効率的に実行するプログラムを作成することで, ハードウェアを意識したプログラミングを体験, 学習する.

組込みシステム向けソフトウェアの 開発言語

C言語	78%
C++言語	15%
アセンブリ言語	4%
Java言語	1%

トロン協会:「組込みシステムにおけるリアルタイムOS
の利用動向に関するアンケート調査報告書」

評価方法

- レポート4回 : 55%
- 小テスト(第4回に実施予定) : 25%
- 平常点(授業中テスト, 回答, ダウンロード) : 20%
- 欠席, 遅刻, 早退は減点

第1回

「ポインタを中心としたC言語の復習」

※配布ファイルはLMS

印刷用配布資料を印刷したものの配布する

メモリ空間(1)

64ビットOSの場合

16進数	アドレス (8バイト, 実体なし)	メモリ (1バイト)	16進数
00000001	00000000.....0000000001	01001101	4d
00000002	00000000.....0000000010	10001110	8e
00000003	00000000.....0000000011	00010101	15
00000004	00000000.....0000000100	01111011	7b
00000005	00000000.....0000000101	11010001	d1

・
・

型と宣言(1)

```
int i;
```

```
i=2;
```

i int型の_____ 値を

2 int型の_____ 値を

型と宣言(2)

kadai1-1.c

#include <stdio.h> → これ以降は省略

```
int main(void)
```

```
{
```

```
    int i;
```

```
    i=511;
```

```
    printf("Value= _____¥n",_____);    ..... i の値を出力
```

```
    printf("Address=_____¥n", _____); ..... i のアドレスを出力
```

```
    return 0; → これ以降は省略
```

```
}
```

▲変数 i の値, アドレスを表示せよ.

Address	Value

型と宣言(3)

```
int main(void)
{
    int i;

    i=511;

    printf("Value= %d¥n",*( i のアドレス));

}
```

***** : 間接演算子 (間接参照演算子, ポインタ演算子)

型と宣言(4)

質問: * の対象になぜ型の情報が必要なのか？

型と宣言(5)

▲変数 **i** に関するメモリマップを作成する.

※可能な限り印刷して右表に追記する.

印刷困難な場合は
memory_space.xlsx利用.

pointer	Address	Value	variable

```
int main(void)
{
    int i=511;
    {
        int j;
        for(j=0;j<sizeof(i);j++)
            printf("%p %02x¥n",((char *)&i+j), *((char *)&i+j));
    }
}
```

(char *): キャスト演算子(型の変更に利用)
この場合は「char型へのポインタ型」に変換

型と宣言(6)

int i; の意味

(1) メモリ領域の確保 (sizeof(int)バイト)

(2) メモリの解釈の仕方の決定 → 型

式中の i と解釈

即ち, アドレス&iからsizeof(int)バイト分のメモリ
をint型として解釈する

※宣言後は, 先頭アドレスと解釈の仕方(型)が解れば良い.
即ち, ポインタ &i があれば i を参照できる.

ポインタ(1)

ポインタは、

(1) アドレスを値として持つ.

(2) 他の型から派生し、型(メモリの解釈の仕方)を持つ.

「ポインタの値を先頭アドレスとして
ポインタの指す型として解釈(変換)すべきもの」

- ・int型へのポインタ
- ・char型へのポインタなどなど

※ 基本的に内容を変更するもの(変数)に対して存在する

ポインタ(2)

(int *) アドレスA

→アドレスAを先頭アドレスとしてint型として_____.

*(int *) アドレスA

→「解釈すべきもの」をそのとおり「解釈した結果」

即ちアドレスAを先頭アドレスとしてint型として_____.

int *pi; と宣言した場合

pi → (int *)(piの値)

→ (piの値)を先頭アドレスとしてint型として_____.

*pi → *(int *)(piの値)

→ (piの値)を先頭アドレスとしてint型として_____.

ポインタ(3)

- 「ポインタ型」も一つの型

- ポインタ型のサイズは？

- メモリの解釈の仕方は？

- ポインタには、ポインタ型の「変数」と「定数」がある

- int型へのポインタ型の変数例
_____； と宣言した時の pi

- int型へのポインタ型の定数例
int i; と宣言した時の
あるいは _____ (アドレスの値)

ポインタ(4)

kadal-4.c

▲ **i** と **pi** に関するメモリマップを作成せよ.

```
int main(void)
{
    int i;
    int *pi;

    i=-1;
    pi=&i;
    printf(“%d¥n”,*pi);

    {省略}
}
```

[illegible]

配列(1)

- 配列の宣言

`int ai[N];` N:定数(コンパイラにより変数も可)

- $(\text{sizeof}(\text{int}) * N)$ バイトのメモリが連続して確保される
- `sizeof(ai)`, `sizeof(int [N])`は何れも同じ配列のサイズ
- では, 配列の各要素のメモリ内での順番は?

配列(2)

▲配列の各要素のメモリ内での配置を作成せよ

```
int main(void)
{
    int ai[3]={0,1,2};
    int n;

    for(n=0;n<3;n++)
        printf("ai[%d]=%d\n",n,ai[n]);

    for(n=0;n<3;n++)
        printf("&ai[%d]=%p\n",n,&ai[n]);
}
```

[illegible]

配列(3)

- ・配列に関するルール1

.... 式中の $ai[n]$ は $*(ai + n)$ と解釈される

- ・配列に関するルール2

.... 配列全体を指すもの(配列名 ai) は, 式中では
配列の先頭要素へのポインタに読み替えられる

$ai \rightarrow \&ai[0]$

即ち, $ai[n] \rightarrow *(\&ai[0] + n)$ と解釈され, ポインタ演算を用いて各要素の先頭アドレスを求め, アクセスする.

配列(4)

▲ $a_i[n]$ は $*(a_i + n)$ と解釈されるとすると、以下の表現は可能か？

- $n[a_i]$
- $(a_i+n)[0]$
- $0[a_i+n]$

配列(5)

・ポインタ演算

+: 普通の足し算

+^p: ポインタ演算としての足し算
(このページ内)

X型へのポインタ +^p 整数



+ 整数 × sizeof(X型)となる

・例えば, ai がint型へのポインタ(変数)の時

$ai +^p n \rightarrow ai +$

・従って, $*(\&ai[0] +^p n)$ が配列のn番目の要素の値を意味する

例: $ai[2] \rightarrow *(\&ai[0] +^p 2) \rightarrow *(\&ai[0] + 8)$

配列(6)

- 配列の動的な生成法

```
int *dai;
```

```
dai=(int *)malloc(N*sizeof(int));
```

(1)必要なサイズのメモリが
連続して確保される

(2)先頭要素へのポインタ

ポインタの型により使用法が決まる

int ai[N]; と基本的に同じ.

→ dai[n]で各要素にアクセス可.